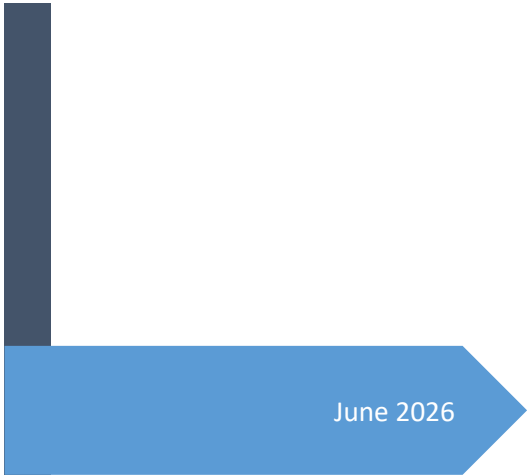
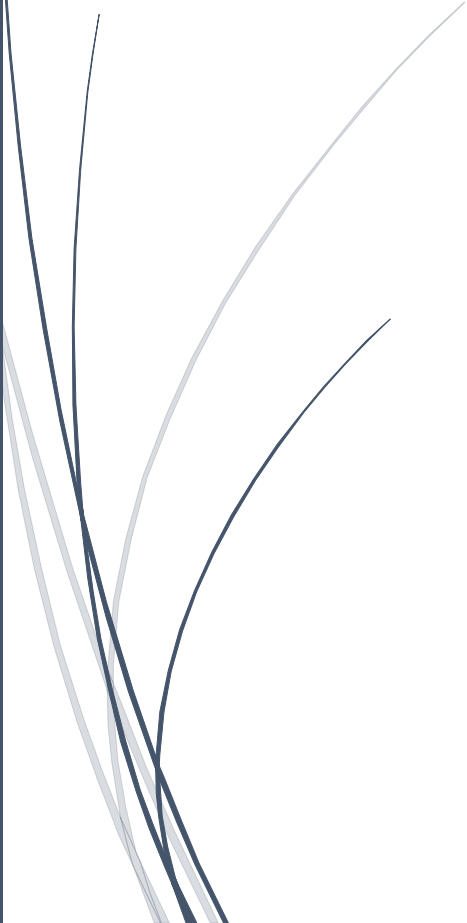


A dark blue vertical bar runs down the left side of the page. A blue arrow points to the right from the bar, containing the date.

June 2026

API SECURITY RISK ANALYSIS REPORT

**FUTURE INTERNS - CYBER SECURITY
TASK 3**

Several thin, curved lines in dark blue and light grey originate from the bottom left and sweep upwards and to the right.

SEDO Kokou Henoc
FUTURE INTERNS

TABLE OF CONTENTS

1. Executive Summary	Page 3
2. Technical Tools & Testing Methodology	Page 3
3. API Security Gaps & Practical Case Studies	Page 3
○ 3.1 <i>Risk 1: Broken Object Level Authorization (BOLA)</i>	
○ 3.2 <i>Risk 2: Lack of Rate Limiting (Brute Force & DoS)</i>	
○ 3.3 <i>Risk 3: Excessive Data Exposure</i>	
4. Remediation & Mitigation Matrix	Page 5
5. Conclusion & SaaS Security Fundamentals	Page 5

1. Executive Summary

This risk analysis report investigates critical security vulnerability gaps within application program interfaces (APIs). As web apps and SaaS ecosystems heavily rely on backend API endpoints, assessing structural flaws such as authorization defects and improper data sanitization is essential to securing corporate and user assets.

2. Technical Tools & Testing Methodology

The security assessments were simulated and mapped using industry-standard tools:

- **Postman / Insomnia:** Used for crafting custom HTTP requests, manipulating header tokens, and analyzing server JSON responses.
- **Browser Developer Tools (Network Tab):** Used to intercept live API payloads transmitted during client-server interactions.
- **Sample Public Test APIs:** Leveraged dummy JSON endpoints to model real-world vulnerabilities safely.

3. API Security Gaps & Practical Case Studies

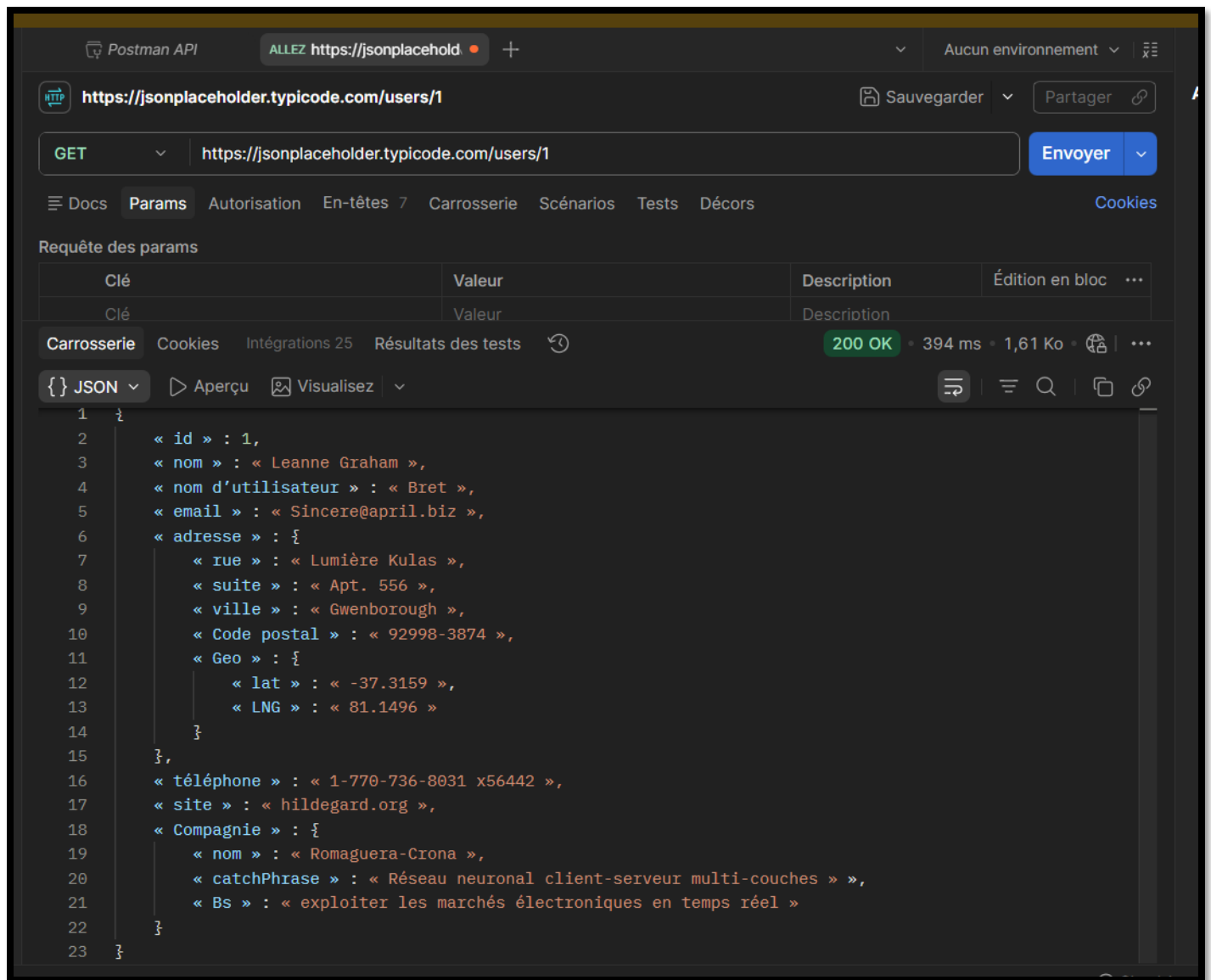
3.1 Risk 1: Broken Object Level Authorization (BOLA / OWASP API1)

- **Description:** The API endpoint allows an authenticated user to access records belonging to another user simply by altering a resource identifier variable in the request path.

❖ Vulnerable **Endpoint Test:**

- *Original Request:* GET /api/v1/accounts/user?id=7701 (Returns user 7701 data)
- *Manipulated Request:* GET /api/v1/accounts/user?id=7702 (Returns user 7702 private data without proper validation)

```
// Attacker payload response intercepted via Postman
{
  "status": "success",
  "account_id": "7702",
  "owner_name": "John Doe",
  "corporate_email": "johndoe@target-client.com",
  "api_key_token": "secret_live_token_99a8b2 "
}
```



3.2 Risk 2: Lack of Rate Limiting (OWASP API4)

- **Description:** The API gateway fails to throttle or limit the frequency of incoming requests targeting sensitive authentication systems.
- **Simulation Matrix:** An automation script executed 150 login attempts within 10 seconds. The server responded with HTTP status 200 OK or 401 Unauthorized for all attempts, instead of issuing an HTTP 429 Too Many Requests restriction.
- **Business Risk:** Exposes corporate portals to automated credential stuffing, brute-force attacks, and application-layer denial of service (DoS).

3.3 Risk 3: Excessive Data Exposure (OWASP API3)

- **Description:** The backend framework automatically serializes full database objects and relies on front-end code to filter out sensitive attributes before rendering them.

- **Interception Scenario:** Inspecting the browser's Network Tab during a standard profile query shows that fields like `password_hash` or `is_admin: true` are sent inside the raw network packets, even though they aren't visible on the webpage text.

4. Remediation & Mitigation Matrix

Identified API Gap	Risk Level	Impact Scope	Mitigation Strategy
Broken Object Authorization	High	Complete client account data hijacking	Enforce object-level access control tokens checking user claims against requested IDs.
Lack of Rate Limiting	Medium	Server downtime & automated brute force	Deploy API gateway rate-limiting (e.g., maximum 60 requests per minute/IP).
Excessive Data Exposure	Medium	Information disclosure & privilege escalation	Custom data transfer objects (DTO) to filter backend properties.

5. Conclusion & SaaS Security Fundamentals

Securing cloud native APIs requires a policy of zero-trust. Developers must treat all client-supplied inputs as hostile, validate tokens strictly at the server level, and ensure information minimality when building system response models.